



```
1 #include <stdio.h>
2
3 int main() {
4     // Write C code here
5     printf("Hello World!");
6
7     return 0;
8 }
9
```

C PROGRAMMING

Presenter

Usha Rani Sargurunathan

Project Engineer

Software Technology Group



KEY TERMINOLOGIES

- What is computer?
- What is program ?
- Programming languages and needs
- Syntax
- Algorithm
- Language Classification : low level $\Rightarrow$ high level
- Translators
- Introduction to c programming.

COMPUTER AND PROGRAM

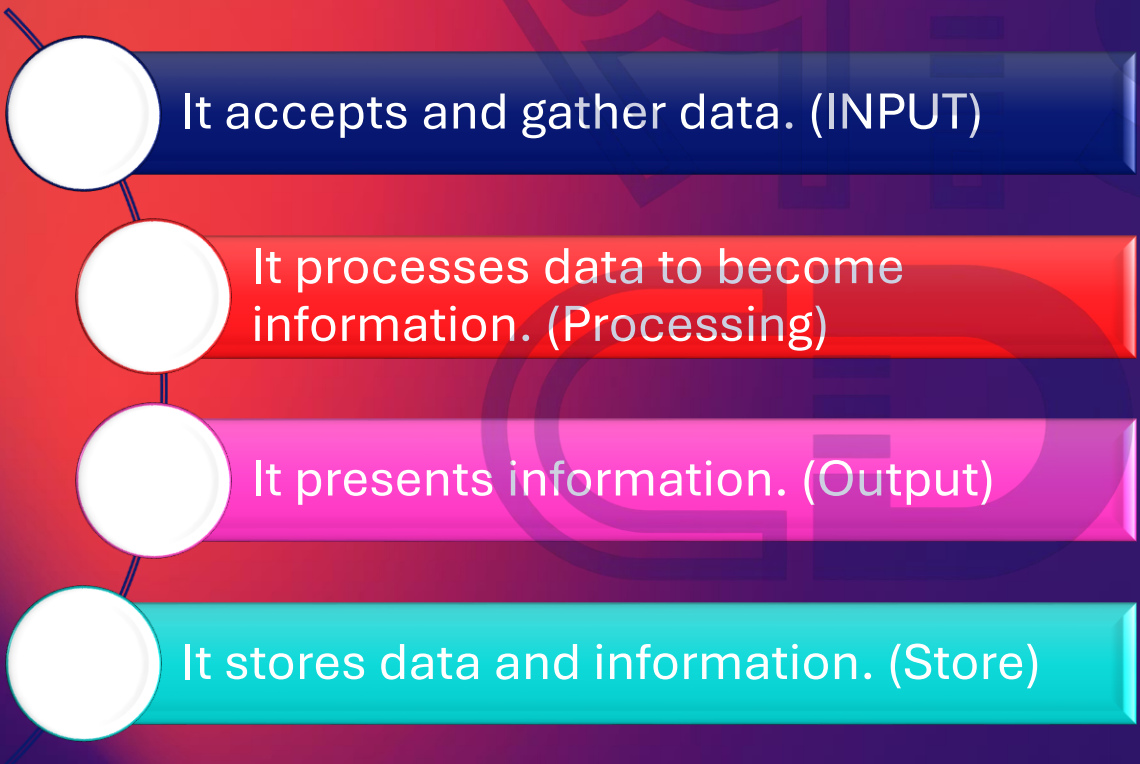


COMPUTER

The word computer comes from the word “compute”, which means, “to calculate”.

A computer is an electronic device that processes data and performs tasks based on the given instructions and delivers the result.

COMPUTER

A vertical diagram on the left side of the slide. It consists of four white circles arranged vertically, connected by a thin line. Each circle is positioned to the left of a horizontal bar of a different color. The bars are dark blue, red, pink, and light blue from top to bottom. Each bar contains text describing a function of a computer.

It accepts and gather data. (INPUT)

It processes data to become information. (Processing)

It presents information. (Output)

It stores data and information. (Store)

Every basic computer program follows the IPO (Input-Process-Output) cycle:

- Accept the data
- Process or manipulate the data
- Display or store the result in the form of human understanding
- Store the data, instructions and results.

But how do you communicate?

WHAT IS A PROGRAM?



Set of instructions given to the machine to do specific task.

Computers cannot think on their own, so they rely on these step-by-step "recipes" to process data and produce an output.

Computer is run by programs.

E.g : Calculator (Mathematical Operations)

Your specific task your program.

SOFTWARE



Definition: Software is a collection of programs, data, or instructions that tell a computer how to perform specific tasks.

Importance: Without software, hardware is non-functional.

Examples: Web browsers, operating systems, video editing tools, and antivirus programs.

TYPES OF SOFTWARE

System Software vs. Application Software

System Software

- Runs the computer itself
- Works in the background
- Example: Windows, macOS, Linux

Application Software

- Helps the user do tasks
- You use it directly
- Example: MS Word, VLC Player,

Think of your computer as a car:

System software
= the engine



Application software =
the radio, GPS, AC

There are two main types of software:

System Software – Manages hardware and provides a platform for running applications.

Application Software – Helps users perform specific tasks like word processing, browsing, and entertainment.

KEY DIFFERENCES

Feature	System Software	Application Software
Purpose	Manages system resources	Helps users perform tasks
Runs on	Background (low-level)	Foreground (high-level)
Dependency	Needed for system functionality	Runs on system software
Examples	Windows, Linux, macOS	MS Word, Photoshop, Chrome



PROGRAMMING LANGUAGE

Humans use natural language (like Hindi/English) to communicate.



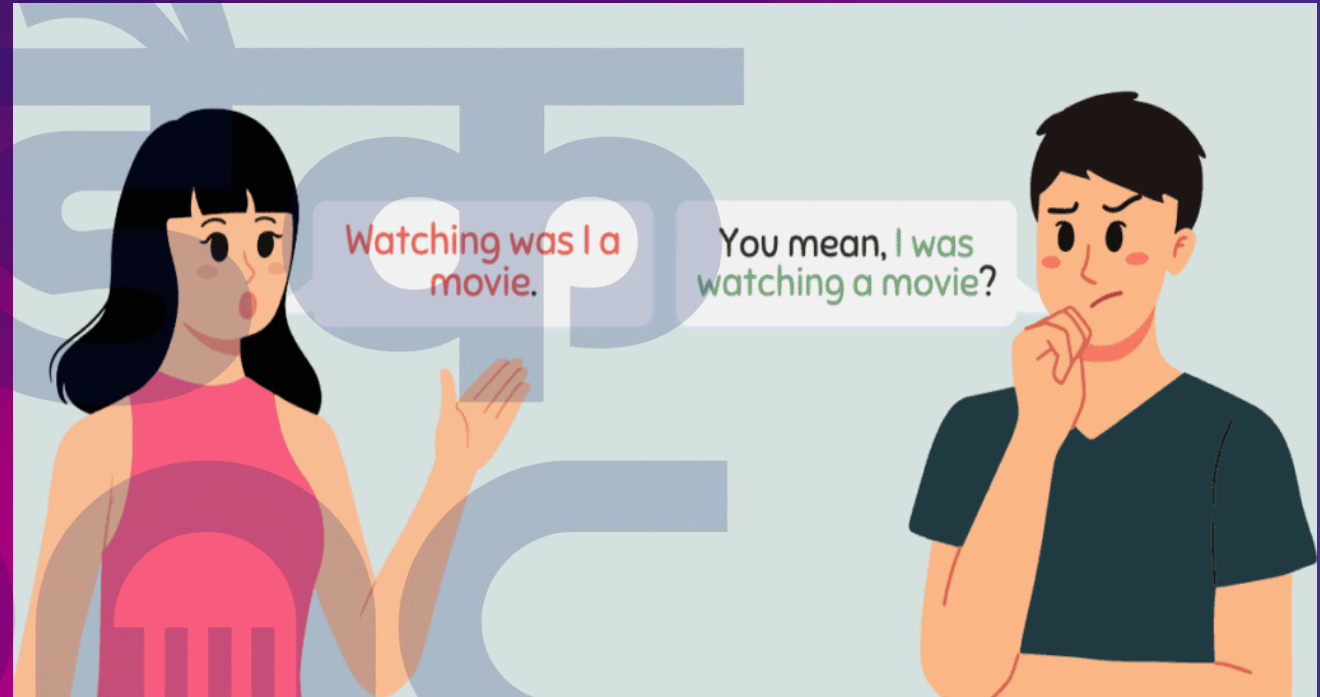
ALGORITHM

An algorithm is a step-by-step procedure for solving a problem or performing a task.



SYNTAX

1. Structure of words in a sentence.
2. Rules of the language.
3. For programming exact syntax must be followed.



PROGRAMMING LANGUAGE



- ✓ A programming language is a set of instructions and syntax used to create programs.
- ✓ Computer understands 0s and 1s only.
- ✓ Program has to be converted into Machine language.
- ✓ CPU perform all task.
- ✓ Before invention of programming language what they use to write a program ?

PROCESSING FLOW

```
1010101011010101010101  
0101010100101001010010  
0101001010100000000000  
0000000000000000000000  
0000000010010011011111  
1111111111111100000000  
0000000000000000000000  
0000000000000000000000  
0000000011111111011111
```

(machine code)



(CPU)

Example : 1 + 2 (Translated into a specific sequence of bits)

0	1	1	0	1	1	1	0	1	1	0	0	0	1	1	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

CLASSIFICATION OF LANGUAGES

A **low-level language** is a programming language that provides no abstraction from the hardware, and it is represented in 0 or 1 forms, which are the machine instructions.

Example :

Machine-level language ,
Assembly language

A **high-level language** is a programming language designed for ease of use, providing abstraction from hardware to simplify problem-solving and development.

Examples:

Java, C++, Ruby, JavaScript,
Python

LOW LEVEL LANGUAGE

Machine Language

- ❖ No abstraction from the hardware.
- ❖ Represented in 0 and 1 forms.
- ❖ Fast execution.
- ❖ Not understood by Humans
- ❖ Extensive knowledge of system architecture required.
- ❖ System dependent.



LOW LEVEL LANGUAGE

Assembly Language

- ❖ A mnemonic is an acronym for an assembly language instruction.
- ❖ Strong binding with machine language.
- ❖ Human readable.
- ❖ Slower than machine language.
- ❖ Machine dependent.
- ❖ Not portable.

Example :

```
MOV AX, 5  
ADD AX, 3  
SUB AX, 3
```

Hardware

```
10101010110  
10101010101  
0101010100
```

Assembly
language

HIGH LEVEL LANGUAGE

- ❖ Portable.
- ❖ Not machine dependent.
- ❖ Sometimes OS dependent.
- ❖ Far from machine code or hardware.
- ❖ Understandable by humans.
- ❖ It is more like English language.
- ❖ Deals with variable , constants , alphabets , numbers rather dealing with CPU architecture.

Hardware

```
10101010110
10101010101
0101010100
```

Assembly
language

High level
languages

LANGUAGE TRANSLATORS

1 Assembler

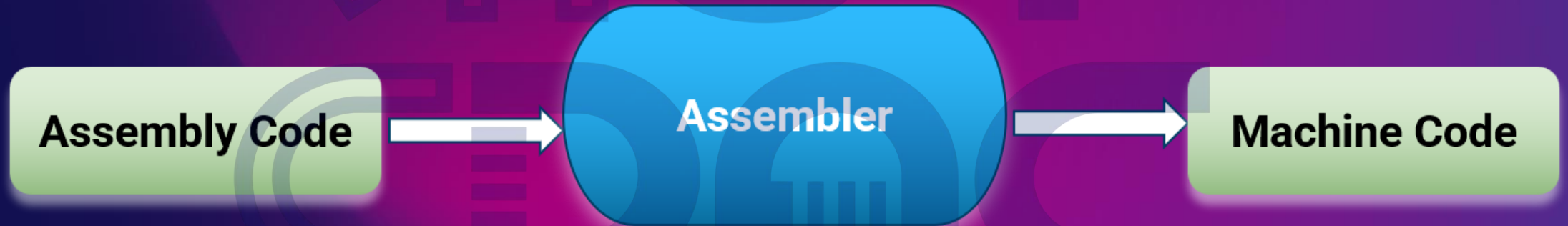
2 Interpreter

3 Compiler

WHY?

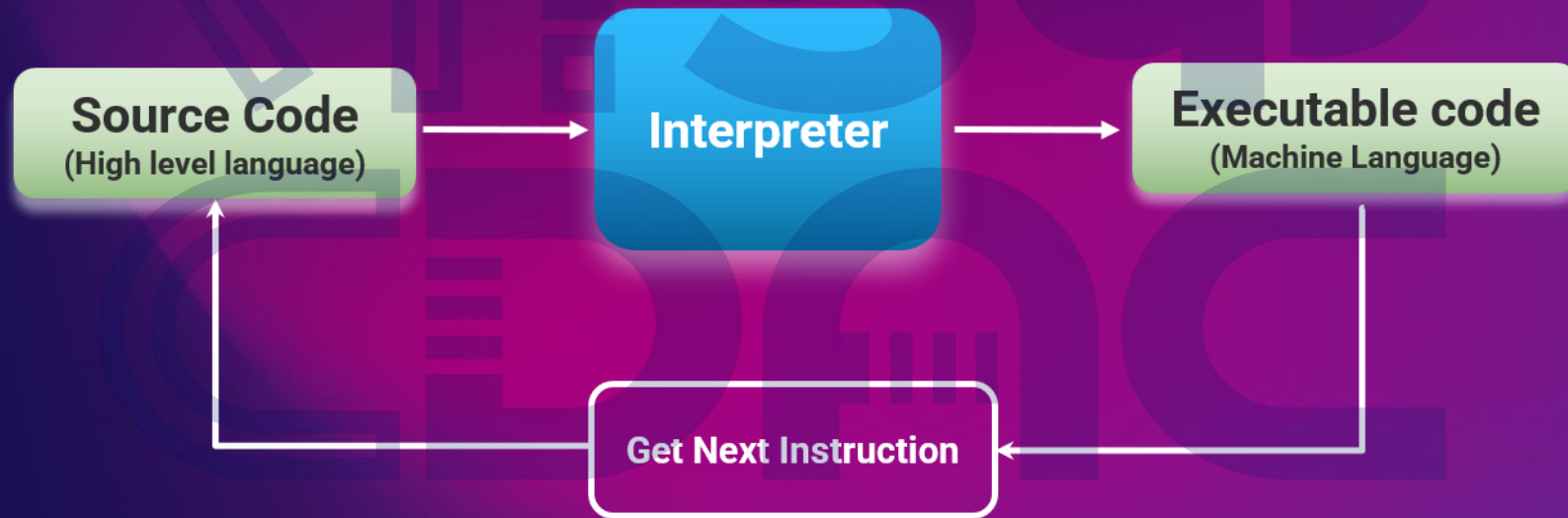
ASSEMBLER

Converts assembly language into machine code (binary).



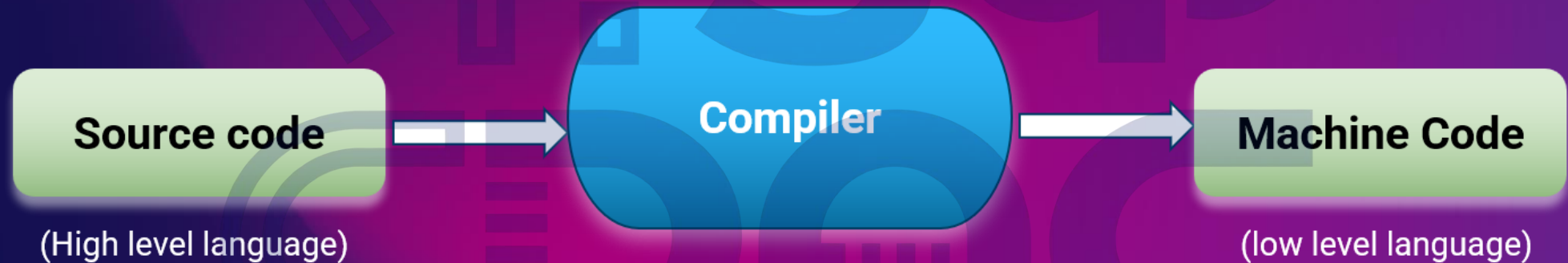
INTERPRETER

Converts high-level language code into machine code, line by line and executes it immediately.



COMPILER

Software that converts program written in a high-level language to a low-level language (Object/Target/Machine Language/0, 1's).



DIFFERENCE IN COMPILER AND INTERPRETER

COMPILER

- ❖ It convert before execution of program.
- ❖ Not parallel process.
- ❖ It will create an executable file (Obj file).



INTERPRETER

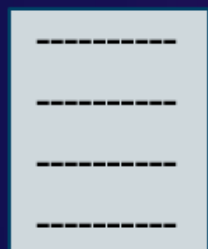
- ❖ It converts while program is running.
- ❖ Parallel process.
- ❖ Every time source code is required.



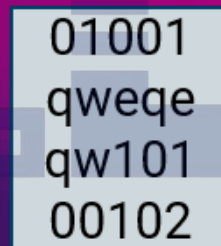
DIFFERENCE IN COMPILER AND INTERPRETER

COMPILER

- ❖ The time taken is very less.
- ❖ Debugging is tough.
- ❖ No need of source code.
- ❖ It creates separate object file and then execute it so it takes more memory.



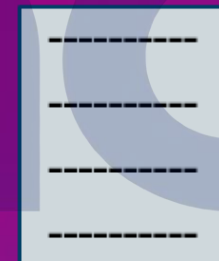
Src code



Obj code

INTERPRETER

- ❖ It is slower.
- ❖ Debugging is easy.
- ❖ Needs source code.
- ❖ It directly runs the program so it takes less memory.



Src code



C PROGRAMMING LANGUAGE

C is a high-level and general-purpose programming language that is ideal for developing portable applications.



HISTORY

- Developed in 1972 by **Dennis Ritchie** at Bell Labs to create the UNIX operating system.
- Before C the language was developed is B language (developed by Ken Thompson as a simplified version of BCPL).
- C was developed to write OS.(UNIX)
- Many software has written in C language. (Embedded system).
- 1980 most widely used programming language.
- 1989 ANSI standardized C as ANSI C.

High-level language

Rich Library

Portable

Middle level language

Features

Pointers

Extensible

Modularity

Dynamic memory allocation

APPLICATIONS



- ❖ System programming : iot devices, routers
- ❖ OS development : kernel, windows
- ❖ Device drivers : keyboard , network adapter, printers
- ❖ Compilers and interpreters : GCC, python interpreter

SOURCE CODE: HELLO WORLD PROGRAM

// - single line comment

/* This comment contains some code which will not be executed. printf("Code enclosed in Comment"); */

/* my first program in C */

Header File

#include <stdio.h>

Main function

int main()

{

Print statement

printf ("Hello World\n");

Return statement

return 0 ;

}

Executable part:

Executable part contains the main function and the respective parenthesis and the control statements that are defined to produce a particular output

COMMANDS

```
cmd> gcc filename.c
```

ELF = Executable and Linkable Format (creates binary executable file)

```
cmd> ./a.out
```

```
cmd> gcc filename.c -o outputFileName
```

```
cmd> ./outputFileName
```

```
cmd> gcc filename.c -o outputFileName
```

```
cmd> ./outputFileName
```

```
cmd> objdump -d myfileopt
```

objdump is a tool used to disassemble binaries.

The -d flag tells it to disassemble the myfileopt binary and show assembly instructions



Questions

MCQs ?

1. Who is the father of C language?

- a) Steve Jobs
- b) James Gosling
- c) Dennis Ritchie
- d) Rasmus Lerdorf

c) Dennis Ritchie

2. Which of the following statements is **true** about computers?

- a) A computer understands only binary language (0s and 1s)
- b) A computer can understand high-level languages directly
- c) A computer does not require an operating system to function
- d) Computers store all data in decimal format

a) A computer understands only binary language (0s and 1s)

3. Which of the following is a programming language?

- a) HTML
- b) Python
- c) Windows
- d) Microsoft Word

b) Python

4. Which of the following applications is commonly developed using C?

- a) Web browsers
- b) Operating Systems
- c) Mobile Games
- d) All of the above

d) All of the above

5. C is classified as which type of language?

- A) Only a high-level language
- B) Only a low-level language
- C) Both high-level and low-level
- D) Neither high-level nor low-level

c) Both high-level and low-level

Delhi Centre

6. The main purpose of an assembler is to?

- a) Convert high-level code to machine code
- b) Convert machine code to high-level code
- c) Convert assembly code to machine code
- d) Interpret and execute programs line by line

c) Convert assembly code to machine code

7. What will happen if you execute a C program without a **main()** function?

- a) It will compile and run successfully
- b) It will cause a runtime error
- c) It will not compile
- d) It will display garbage output

c) It will not compile

THANK YOU